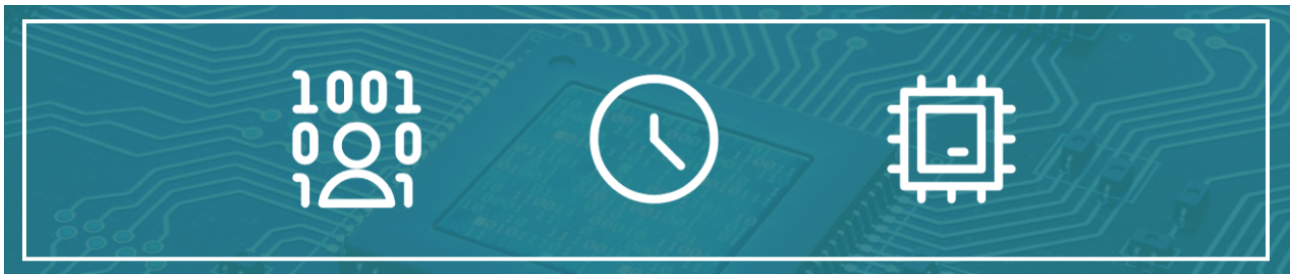
ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

ING4
Groupe 5

RAPPORT DE TP

TP Big Data

Trino : labs 2 et 3



Ruben COMBE--TAMAIN

J'atteste que ce travail est original,
qu'il est le fruit d'un travail individuel
et qu'il a été rédigé de manière autonome.

Paris, le 09/11/2026

Table des matières

1 Environnement et données **3**
1.1 Infrastructure utilisée . . . 3
1.2 Données . . . 3

2 Lab 2 : de silver à gold avec Trino **4**
2.1 Connexion, catalogues et types . . . 4
2.2 Requêtes sur la couche silver . . . 5
2.3 Métadonnées Iceberg . . . 6
2.4 Plan et exécution d'une requête . . . 7
2.5 Couche gold . . . 9
2.6 Validation . . . 11
2.7 Rafraîchissement et time travel . . . 12
2.8 Comparaison avec Spark . . . 13

3 Lab 3 : schéma en étoile **13**
3.1 Dimensions . . . 13
3.2 Table de faits . . . 14
3.3 Requêtes sur l'étoile . . . 15
3.4 Validation . . . 16
3.5 Exercices . . . 18
3.5.1 Exercice 1 : partitionnement . . . 19
3.5.2 Exercice 2 : cohortes par décennie de naissance . . . 19
3.5.3 Exercice 3 : INSERT, MERGE et time travel . . . 20
3.5.4 Exercice 4 : `optimize` . . . 21
3.5.5 Exercice 5 . . . 22
3.6 Stockage des tables gold . . . 22

4 Bilan **22**

5 Annexes **23**
5.1 Catalogue Trino . . . 23
5.2 Scripts Spark . . . 23

I. Environnement et données

A. Infrastructure utilisée

Les labs ont été exécutés sur la **plateforme Adaltas** (Onyxia), dans mon espace `user-r-combe-tamain-ece`. Quatre briques tournent dans ce namespace (Figure 1) :

- le service `vscode-pyspark`, qui lance les commandes et Spark en mode local ;
- **Apache Polaris 1.8.0** et sa base PostgreSQL, déployés avec le chart Helm du lab ;
- **Trino 480** (chart Helm 1.45.0), coordinateur seul ;
- le stockage **S3 de la plateforme**, bucket `user-r-combe-tamain-ece`.

terminal — pods et releases Helm du namespace

```
$ kubectl get pods -o wide ; helm list
NAME  READY  STATUS  RESTARTS  AGE  IP  NODE  NOMINATED  NODE  READINESS  GATES
polaris-5b5d597f8b-qd6x9  1/1  Running  0  25m  10.0.7.33  compute-02  <none>  <none>
polaris-bootstrap-6m2dz  0/1  Completed  0  19m  10.0.7.154  compute-02  <none>  <none>
polaris-postgresql-0  1/1  Running  0  25m  10.0.0.64  compute-03  <none>  <none>
trino-coordinator-788685fb59-2zflb  1/1  Running  0  12m  10.0.0.9  compute-03  <none>  <none>
vscode-pyspark-430512-0  1/1  Running  0  48m  10.0.0.131  compute-03  <none>  <none>

NAME                  NAMESPACE              REVISION       UPDATED                                   STATUS         CHART                 APP VERSION
polaris               user-r-combe-tamain-ece 1             2026-11-09 12:15:15.154745477 +0000 UTC deployed       polaris-1.8.0
trino                 user-r-combe-tamain-ece 1             2026-11-09 12:27:51.650275398 +0000 UTC deployed       trino-1.45.0
vscode-pyspark-430512 user-r-combe-tamain-ece 1             2026-11-09 11:51:22.052294868 +0000 UTC deployed       vscode-pyspark-3.2.7
```

Figure 1 – Pods et releases Helm du namespace (`kubectl get pods`, `helm list`).

Le catalogue Polaris porte le nom du namespace et pointe vers `s3://user-r-combe-tamain-ece/warehouse/` (Figure 2). Trino y accède par le catalogue `lakehouse` (connecteur Iceberg, type REST, OAuth2), défini dans les valeurs du chart Helm (annexe A).

terminal — catalogue Polaris et serveur Trino

```
$ curl $POLARIS_URI/api/management/v1/catalogs | jq ... ; curl http://trino:8080/v1/info
{"name":"user-r-combe-tamain-ece","type":"INTERNAL","storage":"S3","endpoint":"https://s3.seaweedfs.adm.adaltas.cloud","allowedLocations":["s3://user-r-combe-tamain-ece/wareh…

{"nodeId":"trino-coordinator-788685fb59-2zflb","state":"ACTIVE","nodeVersion":{"version":"480"},"environment":"production","coordinator":true,"coordinatorId":"ff7zw","startin…
```

Figure 2 – Catalogue Polaris et état du serveur Trino.

Mode d'exécution. Pour gagner du temps, les étapes préalables (génération des données, silver Spark, déploiement de Polaris et de Trino) et les cellules des labs 2 et 3 ont été enchaînées par un script lancé à l'initialisation du service, au lieu d'être tapées à la main. Les commandes et requêtes sont celles des labs ; le script d'enregistrement de silver dans Polaris est en annexe B. Le cluster étant très chargé, le déploiement de Polaris a duré environ 12 minutes (pods en attente de CPU).

B. Données

Les CSV viennent du générateur du lab `uv` (Faker, graine 42), recopié tel quel. Leur taille correspond exactement à celle donnée dans les labs (7 351 et 331 568 octets, Figure 3). Le jeu contient **50 utilisateurs et 2 829 commandes**.

Écart avec l'énoncé. Les labs annoncent 5 000 commandes. Ce n'est pas ce que produit le générateur : `orders.csv` contient 2 829 lignes, pour 331 568 octets (environ 117 octets par ligne). Tous les comptages de ce rapport sont donc 2 829 et non 5 000.

```
terminal — jeu de données du lab 03

$ wc -c users.csv orders.csv ; head users.csv orders.csv ; aws s3 ls s3://$LAB_BUCKET_NAME/bronze/
  7351 users.csv
331568 orders.csv
338919 total
uuid,username,name,sex,address,mail,birthdate
bdd640fb-0667-4ad1-9c80-317fa3b1799d,garzaanthony,Charles Garcia,M,"908 Jennifer Squares
Robinsonshire, KY 01352",helenpeterson@gmail.com,1935-10-31
17be3111-1a2a-43ed-962b-0f79c37459ee,blairamanda,Ryan Munoz,M,"Unit 6184 Box 9593
DPO AP 09617",stanleykendra@gmail.com,2010-01-26
4
uuid,user_uuid,date,quantity,product
2339ba19-2563-4cc3-8a97-ebf555d596af,bdd640fb-0667-4ad1-9c80-317fa3b1799d,2020-01-01 00:02:17.891488+00:00,4,drink
b49e04cc-c243-49e7-a387-eaf6d4c3a832,bdd640fb-0667-4ad1-9c80-317fa3b1799d,2020-01-01 01:28:08.796458+00:00,5,bread

lignes orders.csv : 2829 | lignes users.csv : 50
2026-11-09 11:51:55     331568 orders.csv
2026-11-09 11:51:53       7351 users.csv
```

Figure 3 – Tailles des CSV, extrait des fichiers, nombre de lignes et contenu du bucket.

La couche silver est construite par PySpark comme dans le lab 07 (mêmes règles : renommage, `trim`/`lower`, découpage de l'adresse, dédoublonnage par fenêtre) et écrite en tables Iceberg dans le catalogue `hadoop lab`. Le journal de l'exécution donne 2 829 commandes, 2 832 avec les 3 doublons ajoutés volontairement, puis 2 829 après dédoublonnage. Le lab 09 enregistre ensuite `silver.orders` dans Polaris sans copier les données, et recrée `silver.users` via Polaris.

Ce qui n'a pas été fait : la partie « Compare with Spark » du lab 2 (la même table gold construite avec Spark) et l'exercice 5 du lab 3. Les dates de naissance diffèrent de l'exemple du lab (version de Faker), sans effet sur les résultats.

II. Lab 2 : de silver à gold avec Trino

A. Connexion, catalogues et types

Problème : se connecter à Trino depuis Python et repérer les tables silver. **Solution :** client `trino` (DB-API) et une fonction `sql()` qui exécute puis récupère les résultats dans un `DataFrame`. **Résultat :** Trino 480, quatre catalogues dont `lakehouse` (`lakehouse`, `system`, `tpcds`, `tpch`), deux tables dans `lakehouse.silver` (Figure 4).

```
python — connexion et catalogues

>>> sql("SELECT version()")
  _col0
0   480

>>> sql("SHOW CATALOGS")
     Catalog
0  lakehouse
1     system
2      tpcds
3       tpch

>>> sql("SHOW SCHEMAS FROM lakehouse")
              Schema
0  information_schema
1             silver
2             system

>>> sql("SHOW TABLES FROM lakehouse.silver")
    Table
0  orders
1   users
```

Figure 4 – Version, catalogues, schémas et tables.

Le schéma `silver` est bien exposé via Polaris. Les types sont en Figure 5.

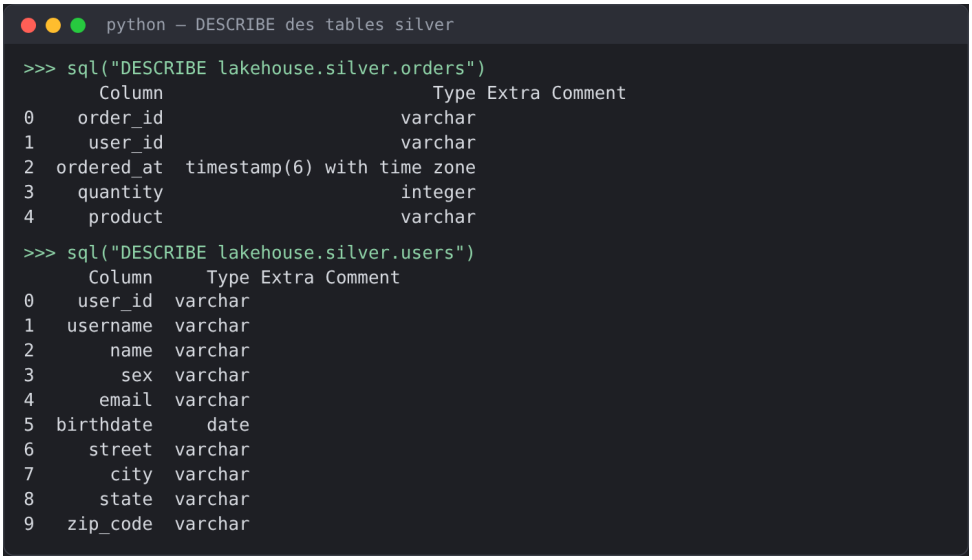


Figure 5 – Types Trino des tables silver.

Que change `catalog=` ? Il fixe le catalogue par défaut de la session : un nom de table non qualifié est cherché dans `lakehouse`. Ici toutes les requêtes sont qualifiées en `catalogue.schéma.table`, donc le paramètre n'a pas d'effet visible.

Type de `ordered_at` ? `timestamp(6) with time zone` : précision à la microseconde, avec fuseau (les valeurs sortent en `+00:00`).

Type de `order_id` ? `varchar`, comme la colonne string du lab Spark.

B. Requêtes sur la couche silver

Problème : vérifier que silver est cohérente avant de construire gold. **Solution :** comptages, répartition par état (analyse du découpage d'adresse) et recherche des commandes sans utilisateur (`LEFT JOIN` + `IS NULL`). **Résultat :** Figure 6.

python — requêtes sur la couche silver

```
>>> sql("SELECT count(*) AS orders FROM lakehouse.silver.orders")
   orders
0    2829

>>> sql("SELECT count(*) AS users FROM lakehouse.silver.users")
   users
0     50

>>> sql("""
...     SELECT state,
...            count(*) AS users,
...            count_if(city IS NULL) AS missing_city
...     FROM lakehouse.silver.users
...     GROUP BY state
...     ORDER BY users DESC
...     LIMIT 10
... """)
  state  users  missing_city
0    FM      3             0
1    AE      3             3
2    VI      2             0
3    AP      2             2
4    NM      2             0
5    VT      2             0
6    TX      2             0
7    PA      2             0
8    HI      2             0
9    NJ      2             0

>>> sql("""
...     SELECT count(*) AS orphan_orders
...     FROM lakehouse.silver.orders AS o
...     LEFT JOIN lakehouse.silver.users AS u ON u.user_id = o.user_id
...     WHERE u.user_id IS NULL
... """)
   orphan_orders
0              0
```

Figure 6 – Comptages, états et commandes orphelines.

- 2 829 commandes et 50 utilisateurs : 50 est le nombre attendu par le lab Spark.
- Les états `AE` (3 utilisateurs, 3 sans ville) et `AP` (2, 2 sans ville) sont des adresses militaires (`APO`/`DPO`) sans ville : le `NULL` sur `city` est voulu.
- **0 commande orpheline** : chaque `user_id` de `orders` existe dans `users`.

Quel test dbt cette requête remplace-t-elle ? Le test `relationships` (clé étrangère `orders.user_id` vers `users.user_id`). Pour l'exécuter à chaque rafraîchissement, on l'enchaîne après la transformation dans un job qui échoue si `orphan_orders > 0`.

Que dit l'absence de session ou de job ? La requête est exécutée immédiatement par le coordinateur, sans soumission : Trino convient aux requêtes interactives ou ad hoc, alors que Spark a un coût de démarrage de session.

C. Métadonnées Iceberg

Les tables cachées `$snapshots`, `$history` et `$files` donnent l'historique et les fichiers (Figures 7 et 8).

python — métadonnées Iceberg de silver.orders

```
>>> sql("""
...     SELECT snapshot_id, committed_at, operation
...     FROM lakehouse.silver."orders$snapshots"
...     ORDER BY committed_at
... """)
          snapshot_id                committed_at  operation
0  8407732450608481646 2026-11-09 11:52:46.239+00:00  overwrite

>>> sql('SELECT * FROM lakehouse.silver."orders$history"')
              made_current_at          snapshot_id parent_id  is_current_ancestor
0 2026-11-09 11:52:46.239+00:00  8407732450608481646      None                 True
```

Figure 7 – Snapshots et historique de `silver.orders`.

Combien de snapshots, écrits par quel moteur ? 1 seul snapshot, de type `overwrite`, écrit par **Spark** (le `createOrReplace` du lab 07). L'exemple du lab en montre 3, car il ajoute des écritures ultérieures que je n'ai pas faites.

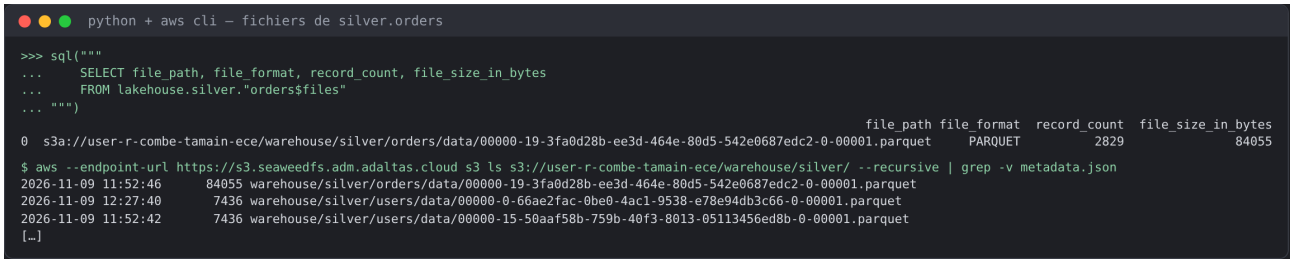


Figure 8 – Fichiers de `silver.orders` : `$files` et `aws s3 ls` (lignes `/data/`).

`aws s3 ls` contre `$files` ? On retrouve le même fichier Parquet unique de `silver.orders` (2 829 lignes, 84 055 octets) ; le listing S3 montre en plus les deux fichiers de `silver.users`, celui de Spark et celui recréé via Polaris. `$files` n'exige **aucun accès au bucket de ma part** : c'est le serveur Trino qui lit les métadonnées avec ses propres identifiants, localisées par Polaris. `aws s3 ls` utilise mes identifiants S3.

D. Plan et exécution d'une requête

La requête étudiée joint `orders` et `users` et compte les commandes par état.

```
python — EXPLAIN (TYPE DISTRIBUTED)

>>> plan = sql("""
...     EXPLAIN (TYPE DISTRIBUTED)
...     SELECT u.state, count(*) AS orders
...     FROM lakehouse.silver.orders AS o
...     JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
...     GROUP BY u.state
... """)
... print(plan.iloc[0, 0])
Trino version: 480
Fragment 0 [HASH]
    Output layout: [state, count]
    Output partitioning: SINGLE []
    Output[columnNames = [state, orders]]
    │   Layout: [state:varchar, count:bigint]
    │   Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
    │   orders := count
    └─ Aggregate[type = FINAL, keys = [state]]
       │   Layout: [state:varchar, count:bigint]
       │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
       │   count := count(count_4)
       └─ LocalExchange[partitioning = HASH, arguments = [state::varchar]]
          │   Layout: [state:varchar, count_4:bigint]
          │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [1]]
                Layout: [state:varchar, count_4:bigint]

Fragment 1 [SOURCE]
    Output layout: [state, count_4]
    Output partitioning: HASH [state]
    Aggregate[type = PARTIAL, keys = [state]]
    │   Layout: [state:varchar, count_4:bigint]
    │   count_4 := count(*)
    └─ InnerJoin[criteria = (user_id = user_id_0), distribution = REPLICATED]
       │   Layout: [state:varchar]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 4.02kB, network: 0B}
       │   Distribution: REPLICATED
       │   dynamicFilterAssignments = {user_id_0 -> #df_314}
       ├─ ScanFilter[table = lakehouse:silver.orders$data@8407732450608481646, dynamicFilters = {user_id = #df_314}]
       │      Layout: [user_id:varchar]
       │      Estimates: {rows: 2829 (22.36kB), cpu: 22.36k, memory: 0B, network: 0B}/{rows: 2829 (22.36kB), cpu: 22.36k, memory: 0B, network: 0B}
       │      user_id := 2:user_id:varchar
       └─ LocalExchange[partitioning = SINGLE]
          │   Layout: [user_id_0:varchar, state:varchar]
          │   Estimates: {rows: 50 (4.02kB), cpu: 0, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [2]]
                Layout: [user_id_0:varchar, state:varchar]

Fragment 2 [SOURCE]
    Output layout: [user_id_0, state]
    Output partitioning: BROADCAST []
    TableScan[table = lakehouse:silver.users$data@412529163050570925]
        Layout: [user_id_0:varchar, state:varchar]
        Estimates: {rows: 50 (4.02kB), cpu: 4.02k, memory: 0B, network: 0B}
        state := 9:state:varchar
        user_id_0 := 1:user_id:varchar
```

Figure 9 – `EXPLAIN (TYPE DISTRIBUTED)` de la jointure.

Dans la Figure 9 :

- **3 fragments** : le 0 fait l'agrégation finale, le 1 scanne `orders`, agrège partiellement et fait la jointure, le 2 scanne `users` ;
- **2** `TableScan`, un par table (`ScanFilter` pour `orders`, car un filtre dynamique est appliqué) ;
- les échanges réseau sont les `RemoteSource` (fragment 0 vers 1, fragment 1 vers 2) ;
- **jointure** `REPLICATED` (broadcast) : `users` (50 lignes) est diffusée à tous les nœuds, ce qui évite de redistribuer les 2 829 commandes.

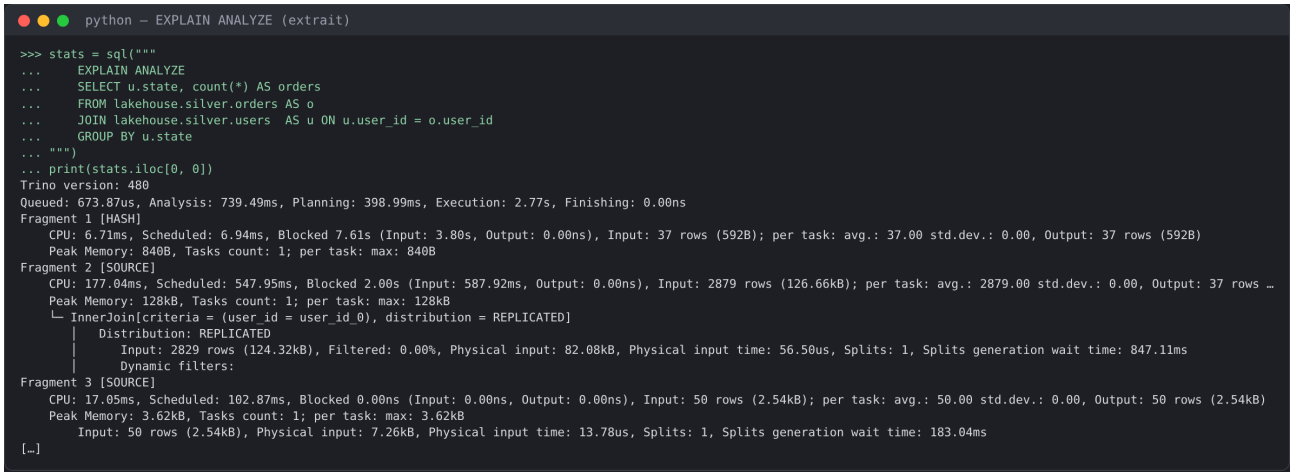


Figure 10 – `EXPLAIN ANALYZE` (extrait des lignes utiles).

Avec `EXPLAIN ANALYZE` (Figure 10), l'exécution fait apparaître un fragment de plus (1 à 3) : le fragment 2 lit 2 879 lignes (2 829 + 50) avec 177 ms de CPU et 128 ko de mémoire de pointe, le fragment 3 lit les 50 utilisateurs. Le scan d'`orders` utilise 1 split, soit un par fichier Parquet (il n'y en a qu'un). Le filtre dynamique ne supprime rien (`Filtered: 0%`), puisque tous les utilisateurs ont des commandes. À cette taille, l'analyse et la planification (739 + 399 ms) représentent environ 40 % du temps d'exécution (2,77 s).

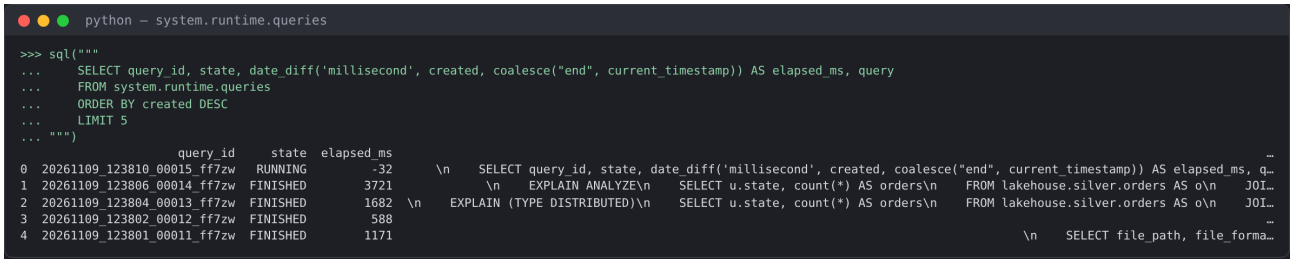


Figure 11 – Historique des requêtes (`system.runtime.queries`).

`system.runtime.queries` est accessible (Figure 11) ; la ligne `RUNNING` est la requête d'historique elle-même. `EXPLAIN (TYPE DISTRIBUTED)` a pris 1,7 s et `EXPLAIN ANALYZE` 3,7 s, qui exécute vraiment la requête. L'interface web de Trino n'a pas été consultée : l'ingress est désactivé dans le chart (accès par `port-forward` uniquement), l'historique ci-dessus la remplace.

E. Couche gold

Trois tables Iceberg sont créées par `CREATE TABLE AS SELECT` dans le schéma `gold` (Figures 12 à 15). Le nombre de lignes écrites est le résultat de chaque `CTAS`.

```
python — orders_by_state_month

>>> sql("CREATE SCHEMA IF NOT EXISTS lakehouse.gold")
Empty DataFrame
Columns: []
Index: []

>>> sql("SHOW SCHEMAS FROM lakehouse")
              Schema
0               gold
1  information_schema
2             silver
3             system

>>> sql("""
...     CREATE TABLE lakehouse.gold.orders_by_state_month AS
...     SELECT
...         u.state,
...         date_trunc('month', o.ordered_at) AS month,
...         count(*)                          AS orders,
...         sum(o.quantity)                   AS units,
...         count(DISTINCT o.user_id)         AS buyers
...     FROM lakehouse.silver.orders AS o
...     JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
...     GROUP BY 1, 2
... """)
   rows
0    50

>>> sql("SELECT * FROM lakehouse.gold.orders_by_state_month ORDER BY month, state LIMIT 10")
  state                     month  orders  units  buyers
0    AL 2020-01-01 00:00:00+00:00      65    199       1
1    AP 2020-01-01 00:00:00+00:00      17     49       1
2    AS 2020-01-01 00:00:00+00:00      12     40       1
3    CO 2020-01-01 00:00:00+00:00      17     43       1
4    FM 2020-01-01 00:00:00+00:00      66    200       1
5    GA 2020-01-01 00:00:00+00:00       4     12       1
6    IN 2020-01-01 00:00:00+00:00      49    175       1
7    KY 2020-01-01 00:00:00+00:00      83    256       1
8    MH 2020-01-01 00:00:00+00:00      21     57       1
9    NE 2020-01-01 00:00:00+00:00      11     30       1
```

Figure 12 – Schéma `gold` et table `orders_by_state_month` (50 lignes).

```
python — top_products

>>> sql("""
...     CREATE TABLE lakehouse.gold.top_products AS
...     SELECT
...         product,
...         count(*)      AS orders,
...         sum(quantity) AS units,
...         rank() OVER (ORDER BY sum(quantity) DESC) AS units_rank
...     FROM lakehouse.silver.orders
...     GROUP BY product
... """)
   rows
0     6

>>> sql("SELECT * FROM lakehouse.gold.top_products ORDER BY units_rank LIMIT 10")
     product  orders  units  units_rank
0  croissant     491   1523           1
1      drink     484   1493           2
2    brioche     468   1424           3
3      bread     458   1420           4
4      donut     462   1390           5
5     cookie     466   1365           6
```

Figure 13 – Table `top_products` (6 lignes).

`croissant` est le produit le plus vendu (1 523 unités), suivi de `drink` (1 493) et `brioche` (1 424).

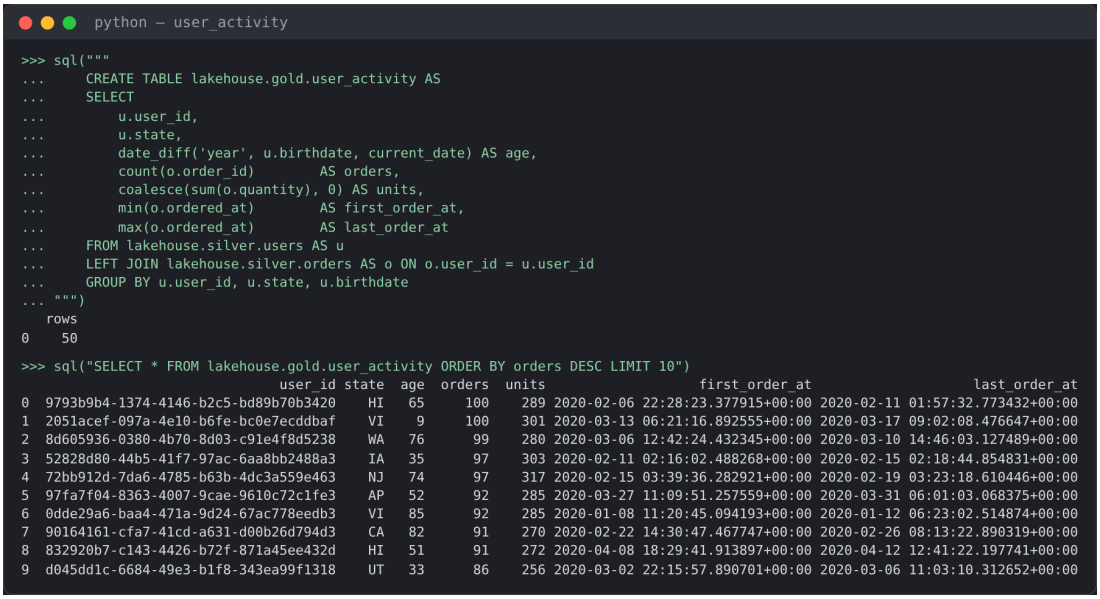


Figure 14 – Table `user_activity` (50 lignes) : les 10 utilisateurs les plus actifs.

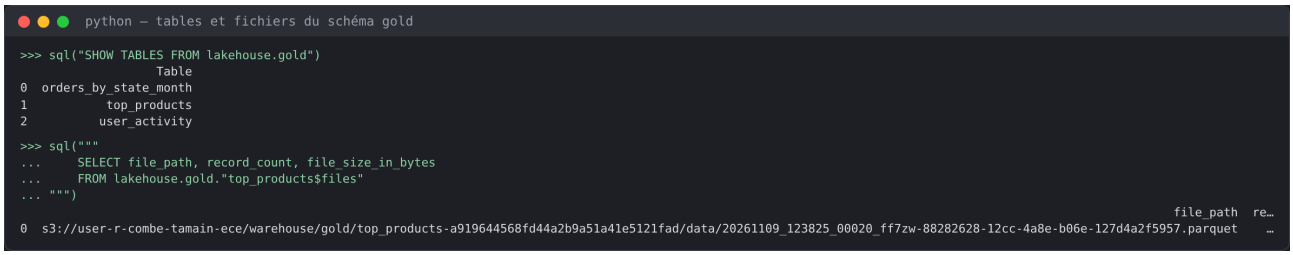


Figure 15 – Tables du schéma `gold` et fichier de `top_products`.

Pourquoi `LEFT JOIN` pour `user_activity` et jointure interne pour `orders_by_state_month` ? `user_activity` doit contenir tous les utilisateurs, y compris ceux sans commande (50 lignes obtenues). `orders_by_state_month` part des commandes : une commande sans utilisateur disparaîtrait silencieusement. C'est donc ce choix de jointure interne qui dépend de la requête orpheline : elle renvoie 0, la jointure interne est sans perte.

Effet de `current_date` ? L'âge change chaque jour, donc `user_activity` est périmée dès le lendemain : elle doit être reconstruite au moins une fois par jour, ou stocker la date de naissance et calculer l'âge à la lecture.

Où sont les fichiers de gold, et qui a décidé ? Sous `s3://user-r-combe-tamain-ece/warehouse/gold/<table` (Figure 30). L'emplacement vient de Polaris : base `default-base-location` du catalogue, plus le nom de la table et un suffixe unique.

F. Validation

Solution : comparer les agrégats de gold à silver avec des `assert`. **Résultat :** aucune `AssertionError` (Figure 16) : 8 615 unités dans silver, dans `orders_by_state_month` et dans `top_products` ; 50 utilisateurs et 2 829 commandes retrouvés dans `user_activity`. La couche gold est cohérente avec silver.

```
python — validation de la couche gold

>>> silver_units = sql("SELECT sum(quantity) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... gold_units = sql("SELECT sum(units) AS v FROM lakehouse.gold.orders_by_state_month").iloc[0, 0]
... product_units = sql("SELECT sum(units) AS v FROM lakehouse.gold.top_products").iloc[0, 0]
... assert silver_units == gold_units
... assert silver_units == product_units
... print(silver_units, gold_units, product_units)
8615 8615 8615

>>> silver_users = sql("SELECT count(*) AS v FROM lakehouse.silver.users").iloc[0, 0]
... gold_users = sql("SELECT count(*) AS v FROM lakehouse.gold.user_activity").iloc[0, 0]
... silver_orders = sql("SELECT count(*) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... gold_orders = sql("SELECT sum(orders) AS v FROM lakehouse.gold.user_activity").iloc[0, 0]
... assert silver_users == gold_users
... assert silver_orders == gold_orders
... print(silver_users, gold_users, silver_orders, gold_orders)
50 50 2829 2829
```

Figure 16 – Validation de gold contre silver.

G. Rafraîchissement et time travel

Solution : vider `top_products` par `DELETE`, la remplir par `INSERT`, puis lire la table au premier snapshot (Figure 17).

```
python — rafraîchissement et time travel

>>> sql("DELETE FROM lakehouse.gold.top_products")
   rows
0     6

>>> sql("SELECT count(*) AS products FROM lakehouse.gold.top_products")
   products
0         0

>>> sql("""
...     INSERT INTO lakehouse.gold.top_products
...     SELECT product, count(*), sum(quantity),
...            rank() OVER (ORDER BY sum(quantity) DESC)
...     FROM lakehouse.silver.orders
...     GROUP BY product
... """)
   rows
0     6

>>> sql("""
...     SELECT snapshot_id, committed_at, operation
...     FROM lakehouse.gold."top_products$snapshots"
...     ORDER BY committed_at
... """)
           snapshot_id                committed_at operation
0  5679923997119223469 2026-11-09 12:38:28.711+00:00    append
1  7721050443174576655 2026-11-09 12:38:45.105+00:00    delete
2   433421330162471359 2026-11-09 12:38:48.706+00:00    append

>>> first = sql("""
...     SELECT snapshot_id
...     FROM lakehouse.gold."top_products$snapshots"
...     ORDER BY committed_at
...     LIMIT 1
... """).iloc[0, 0]
... sql(f"SELECT count(*) AS products FROM lakehouse.gold.top_products FOR VERSION AS OF {first}")
   products
0         6

>>> sql("SELECT count(*) AS products FROM lakehouse.gold.top_products")
   products
0         6
```

Figure 17 – `DELETE`, `INSERT`, snapshots et `FOR VERSION AS OF`.

Combien de snapshots après le rafraîchissement ? 3 : `append` (création), `delete`, `append`. Après le `DELETE`, la table contient 0 ligne ; après l'`INSERT`, 6. Le time travel sur le premier snapshot renvoie bien 6 produits.

Que voit un tableau de bord entre les deux ? Une table **vide** : chaque instruction est un commit séparé. Ici la fenêtre a duré 3,6 s (commit `delete` à 12 :38 :45,105, commit `append` à 12 :38 :48,706),

mais elle existe. Avec un `MERGE`, la mise à jour est un seul commit : le lecteur voit l'ancien état ou le nouveau, jamais un état vide.

H. Comparaison avec Spark

Cette partie n'a pas été exécutée (je n'ai pas reconstruit `top_products` avec Spark). Réponses de principe, tirées de ce que j'ai observé côté Trino et de ce que demande le lab Spark :

- **Besoins au-delà de la requête.** Trino : un client Python (`trino`) et une connexion, rien d'autre. Spark : une session, le jar Iceberg, `hadoop-aws` et des identifiants S3 temporaires.
- **Tableau de bord toutes les minutes :** Trino (latence faible). **Transformation nocturne de 10 To :** Spark (reprise sur perte de partition). **Exploration de silver par un data scientist :** Trino.
- **Tolérance aux pannes.** Si un worker est perdu, Trino fait échouer la requête. C'est acceptable ici (requêtes de moins d'une seconde, relancées sans coût) mais pas pour une transformation de 10 To qui dure des heures et serait à refaire entièrement.

III. Lab 3 : schéma en étoile

A. Dimensions

Problème : modéliser gold en étoile, avec une table de faits et des dimensions. **Solution :** `dim_user` et `dim_product` avec une clé de substitution `row_number()`, et `dim_date` générée par `sequence()` + `UNNEST`, de clé `AAAAMMJJ`. **Résultat :** Figures 18 et 19.

python — dim_user et dim_product

```
>>> sql("""
...     CREATE TABLE lakehouse.gold.dim_user AS
...     SELECT
...         row_number() OVER (ORDER BY user_id) AS user_key,
...         user_id, username, name, sex, birthdate, city, state, zip_code
...     FROM lakehouse.silver.users
... """)
   rows
0    50

>>> sql("""
...     CREATE TABLE lakehouse.gold.dim_product AS
...     SELECT row_number() OVER (ORDER BY product) AS product_key, product
...     FROM (SELECT DISTINCT product FROM lakehouse.silver.orders)
... """)
   rows
0     6

>>> sql("SELECT * FROM lakehouse.gold.dim_user ORDER BY user_key LIMIT 5")
   user_key                               user_id       username             name sex   birthdate           city state zip_code
0         1  09a0a0f6-73b0-4091-b634-c1694f76e388  gilbertwilliam      Donna Newman   F  1971-08-25    Garciamouth    VT    11554
1         2  0b2f6d5c-700b-4d5f-b89f-72f32a60c652        karen73     Nathan Montes   M  2019-03-27     Port James    PA    18641
2         3  0dde29a6-baa4-471a-9d24-67ac778eedb3     wrightjames    Monica Mcbride   F  1941-08-14  Tuckerborough    VI    53066
3         4  1190f938-a66f-47f7-b966-9fa759970043      howelljohn  Tiffany Gonzalez   F  1956-11-12  Port Loriport    CT    76350
4         5  12738a23-5aaa-432f-8e63-22b6ab05347f   atkinsbarbara      Andrew Smith   M  1936-07-22  East Ryanfurt    NC    10809

>>> sql("SELECT * FROM lakehouse.gold.dim_product ORDER BY product_key")
   product_key    product
0            1      bread
1            2    brioche
2            3     cookie
3            4  croissant
4            5      donut
5            6      drink
```

Figure 18 – `dim_user` (50 lignes) et `dim_product` (6 lignes).

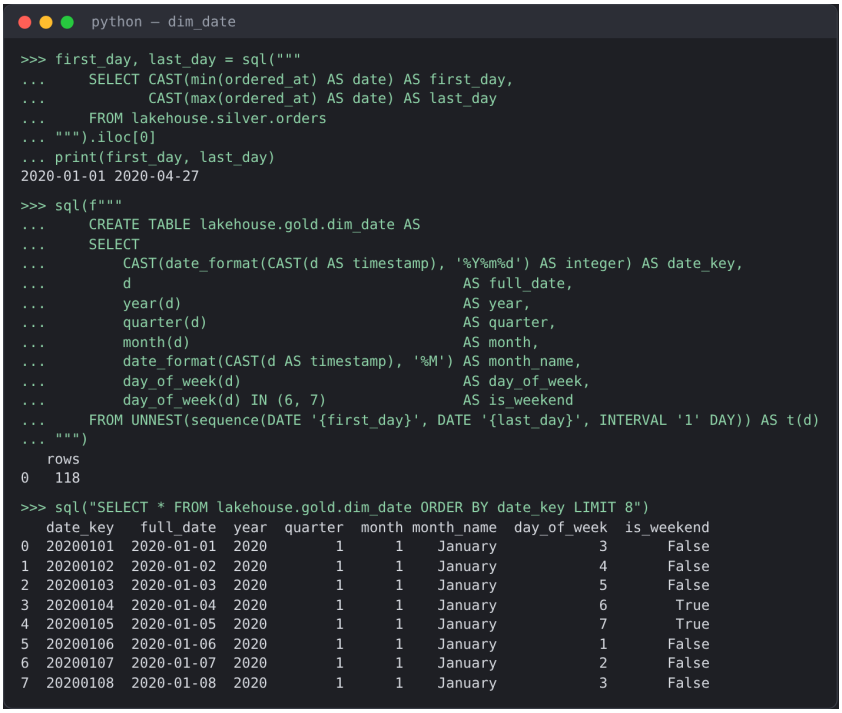


Figure 19 – `dim_date` : 118 jours, du 2020-01-01 au 2020-04-27.

`dim_date` couvre exactement la période des commandes (première et dernière date lues dans silver). Le 4 janvier 2020 est un samedi : `day_of_week` = 6 et `is_weekend` vrai, ce qui valide le calcul.

B. Table de faits

`fact_orders` garde une ligne par commande, avec les clés des dimensions, obtenues par `LEFT JOIN` (Figure 20) : 2 829 lignes, comme `silver.orders`. Les clés sont des `bigint` (`row_number`) et `date_key` un `integer`.

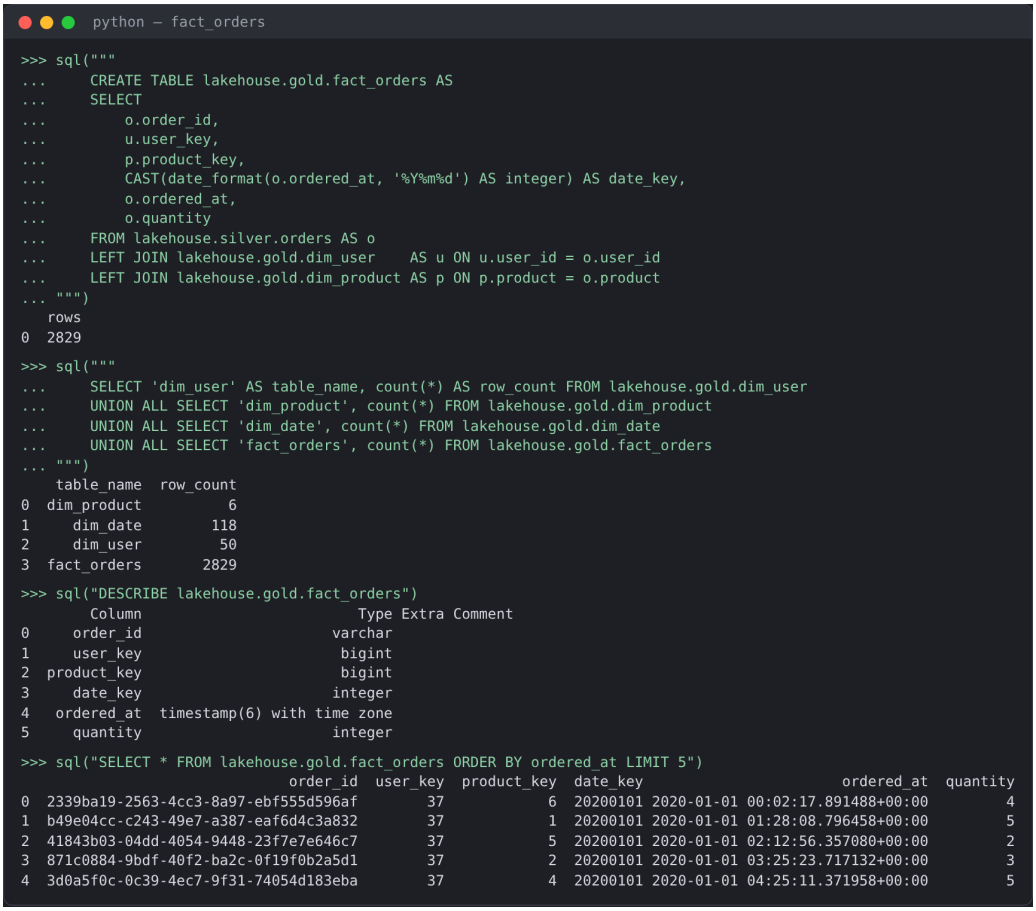


Figure 20 – Comptage des tables de l'étoile, schéma et extrait de `fact_orders`.

C. Requêtes sur l'étoile

Les requêtes de la Figure 21 joignent le fait aux seules dimensions utiles. La seconde reconstruit `orders_by_state_month` sans lire silver ; la comparaison avec la vraie table de gold renvoie `True` : les 10 premières lignes (commandes, unités, acheteurs) sont identiques.

```
python — requêtes sur l'étoile

>>> sql("""
...     SELECT d.year, u.state, p.product,
...            count(*)        AS orders,
...            sum(f.quantity) AS units
...     FROM lakehouse.gold.fact_orders AS f
...     JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
...     JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
...     JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
...     GROUP BY d.year, u.state, p.product
...     ORDER BY units DESC
...     LIMIT 10
... """)
   year state    product  orders  units
0  2020    FM    brioche      43    134
1  2020    VI      bread      37    110
2  2020    HI      bread      35    110
3  2020    VI  croissant      34    107
4  2020    HI  croissant      36    107
5  2020    VI    brioche      33    104
6  2020    FM  croissant      32    102
7  2020    AE      bread      31    100
8  2020    FM      donut      33     98
9  2020    VT      drink      33     96

>>> star = sql("""
...     SELECT u.state, d.year, d.month,
...            count(*)                  AS orders,
...            sum(f.quantity)           AS units,
...            count(DISTINCT f.user_key) AS buyers
...     FROM lakehouse.gold.fact_orders AS f
...     JOIN lakehouse.gold.dim_date    AS d ON d.date_key = f.date_key
...     JOIN lakehouse.gold.dim_user    AS u ON u.user_key = f.user_key
...     GROUP BY u.state, d.year, d.month
...     ORDER BY d.year, d.month, u.state
...     LIMIT 10
... """)
... star
  state  year  month  orders  units  buyers
0    AL  2020      1      65    199       1
1    AP  2020      1      17     49       1
2    AS  2020      1      12     40       1
3    CO  2020      1      17     43       1
4    FM  2020      1      66    200       1
5    GA  2020      1       4     12       1
6    IN  2020      1      49    175       1
7    KY  2020      1      83    256       1
8    MH  2020      1      21     57       1
9    NE  2020      1      11     30       1

>>> ref = sql("SELECT state, orders, units, buyers FROM lakehouse.gold.orders_by_state_month ORDER BY month, state LIMIT 10")
... print(ref.to_string())
... print("Identique a l'etoile :", (ref[["orders","units","buyers"]].values == star[["orders","units","buyers"]].values).all() and (ref.state.values == star.state.values).al…
  state  orders  units  buyers
0    AL      65    199       1
1    AP      17     49       1
2    AS      12     40       1
3    CO      17     43       1
4    FM      66    200       1
5    GA       4     12       1
6    IN      49    175       1
7    KY      83    256       1
8    MH      21     57       1
9    NE      11     30       1
Identique a l'etoile : True
```

Figure 21 – Requêtes sur l'étoile et comparaison avec `orders_by_state_month`.

Comparaison des deux approches. `orders_by_state_month` répond à une seule question, vite (un scan, aucune jointure). L'étoile répond à toute combinaison d'axes (année × état × produit, par exemple) sans nouvelle table, mais coûte des jointures à chaque requête (3 jointures dans la première).

D. Validation

Les trois contrôles de l'énoncé passent sans erreur (Figure 22) : 2 829 commandes et 8 615 unités dans silver et dans le fait, 0 commande sans ligne de dimension complète. J'y ai ajouté un contrôle hors énoncé : en rejoignant le fait à silver par `order_id`, la clé pointe toujours vers le bon utilisateur et le bon produit (0 erreur sur les deux) et les mesures sont identiques.

```
python — validation de l'étoile
>>> silver_orders = sql("SELECT count(*) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... fact_orders = sql("SELECT count(*) AS v FROM lakehouse.gold.fact_orders").iloc[0, 0]
... silver_units = sql("SELECT sum(quantity) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... fact_units = sql("SELECT sum(quantity) AS v FROM lakehouse.gold.fact_orders").iloc[0, 0]
... assert silver_orders == fact_orders, f"{silver_orders} != {fact_orders}"
... assert silver_units == fact_units, f"{silver_units} != {fact_units}"
... unmatched = sql("""
...     SELECT count(*) AS v
...     FROM lakehouse.gold.fact_orders AS f
...     LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
...     LEFT JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
...     LEFT JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
...     WHERE u.user_key IS NULL OR p.product_key IS NULL OR d.date_key IS NULL
... """).iloc[0, 0]
... assert unmatched == 0, f"{unmatched} orders without a complete dimension row"
... print(silver_orders, fact_orders, silver_units, fact_units, unmatched)
2829 2829 8615 8615 0
>>> # Controle supplementaire (hors enonce) : la cle pointe-t-elle vers le BON utilisateur / produit ?
... sql("""
...     SELECT
...       count_if(u.user_id <> o.user_id)  AS wrong_user,
...       count_if(p.product <> o.product)  AS wrong_product,
...       count_if(f.ordered_at <> o.ordered_at OR f.quantity <> o.quantity) AS wrong_measure
...     FROM lakehouse.gold.fact_orders f
...     JOIN lakehouse.silver.orders o ON o.order_id = f.order_id
...     JOIN lakehouse.gold.dim_user u ON u.user_key = f.user_key
...     JOIN lakehouse.gold.dim_product p ON p.product_key = f.product_key
... """)
   wrong_user  wrong_product  wrong_measure
0           0              0              0
```

Figure 22 – Validation de l'étoile (`assert`) et contrôle supplémentaire.

Ces contrôles suffisent-ils ? Non. Compter les lignes et sommer les quantités ne détecte pas une commande rattachée au **mauvais** utilisateur ou au mauvais produit : si deux clés sont permutées, les totaux restent justes. D'où le contrôle supplémentaire ci-dessus (comparaison ligne à ligne avec silver). On pourrait ajouter l'unicité de `order_id` et des clés de dimension, et l'absence de `NULL` sur les clés.

Grain. `fact_orders` : une commande. `dim_date` : un jour calendaire.

Quelle assertion casse selon le défaut ? Testé sans rien écrire (Figure 23) :

- `user_id` **en double dans** `dim_user` : la jointure duplique les commandes de cet utilisateur (2 830 devient 2 930 lignes de fait simulées), donc **la première assertion** (nombre de lignes silver contre fait) échoue.
- **Commande sans produit (`NULL`)** : le nombre de lignes reste correct, mais `product_key` est `NULL` pour 1 ligne (`NULL = NULL` est faux en SQL), donc **l'assertion sur les commandes sans dimension complète** échoue.

python — scénarios de défauts (simulés, sans écriture)

```
>>> # Defaut 1 : un user_id present deux fois dans dim_user (simule, sans ecriture)
... sql("""
...     WITH dim_user_dup AS (
...         SELECT user_key, user_id FROM lakehouse.gold.dim_user
...         UNION ALL
...         SELECT 999, user_id FROM lakehouse.gold.dim_user WHERE user_id = '9793b9b4-1374-4146-b2c5-bd89b70b3420'
...     )
...     SELECT (SELECT count(*) FROM lakehouse.silver.orders) AS silver_orders,
...            count(*) AS fact_orders_simule
...     FROM lakehouse.silver.orders o LEFT JOIN dim_user_dup u ON u.user_id = o.user_id
... """)
   silver_orders  fact_orders_simule
0           2830               2930

>>> # Defaut 2 : une commande sans produit (NULL), simulee sans ecriture
... sql("""
...     WITH orders_null AS (
...         SELECT order_id, product FROM lakehouse.silver.orders
...         UNION ALL SELECT 'x-null-product', CAST(NULL AS varchar)
...     )
...     SELECT count(*) AS orders, count_if(p.product_key IS NULL) AS product_key_null
...     FROM orders_null o LEFT JOIN lakehouse.gold.dim_product p ON p.product = o.product
... """)
   orders  product_key_null
0    2831                1

>>> # Defaut 3 : ajout d'un utilisateur dans dim_user reconstruite avec row_number() (simule)
... sql("""
...     WITH new_dim AS (
...         SELECT row_number() OVER (ORDER BY user_id) AS user_key, user_id
...         FROM (SELECT user_id FROM lakehouse.silver.users
...               UNION ALL SELECT '00000000-0000-0000-0000-00000000000a')
...     )
...     SELECT o.user_id, o.user_key AS cle_actuelle, n.user_key AS cle_apres_reconstruction
...     FROM lakehouse.gold.dim_user o JOIN new_dim n ON n.user_id = o.user_id
...     WHERE o.user_key <> n.user_key ORDER BY o.user_key LIMIT 5
... """)
                              user_id  cle_actuelle  cle_apres_reconstruction
0  09a0a0f6-73b0-4091-b634-c1694f76e388             1                         2
1  0b2f6d5c-700b-4d5f-b89f-72f32a60c652             2                         3
2  0dde29a6-baa4-471a-9d24-67ac778eedb3             3                         4
3  1190f938-a66f-47f7-b966-9fa759970043             4                         5
4  12738a23-5aaa-432f-8e63-22b6ab05347f             5                         6

>>> sql("""
...     WITH new_dim AS (
...         SELECT row_number() OVER (ORDER BY user_id) AS user_key, user_id
...         FROM (SELECT user_id FROM lakehouse.silver.users
...               UNION ALL SELECT '00000000-0000-0000-0000-00000000000a')
...     )
...     SELECT count_if(o.user_key <> n.user_key) AS users_dont_la_cle_change, count(*) AS users
...     FROM lakehouse.gold.dim_user o JOIN new_dim n ON n.user_id = o.user_id
... """)
   users_dont_la_cle_change  users
0                        50     50
```

Figure 23 – Défauts simulés : doublon dans la dimension, produit `NULL`, instabilité des clés (sans écriture).

Clés `row_number()` après ajout d'un utilisateur. Elles ne sont pas stables : en simulant l'ajout d'un utilisateur dont l'identifiant se classe en premier, la clé change pour **les 50 utilisateurs existants** (Figure 23, deux dernières requêtes). Si `dim_user` est reconstruite sans reconstruire `fact_orders`, les commandes pointent vers le mauvais utilisateur : il faut reconstruire le fait avec la dimension, ou utiliser des clés persistantes (hash de `user_id` ou séquence conservée).

Un utilisateur déménage. `dim_user` écrase les attributs : **toutes** ses commandes passées affichent le nouvel état. Garder l'historique demande une dimension à variation lente de type 2 : une ligne par version, avec nouvelle clé de substitution, dates de début et de fin de validité et un indicateur de ligne courante ; le fait référence la version valide à la date de la commande.

E. Exercices

(a) Exercice 1 : partitionnement

`orders_by_state_month_part` est partitionnée par mois (`month(month)`) : 4 partitions et donc 4 fichiers (un par mois), contre 1 fichier pour la table non partitionnée (Figure 24).

```
python — exercice 1 : partitionnement

>>> sql("""
...     CREATE TABLE lakehouse.gold.orders_by_state_month_part
...     WITH (partitioning = ARRAY['month(month)'])
...     AS SELECT * FROM lakehouse.gold.orders_by_state_month
... """)
   rows
0    50

>>> sql("""SELECT partition, record_count, file_count FROM lakehouse.gold."orders_by_state_month_part$partitions" ORDER BY partition LIMIT 5""")
          partition  record_count  file_count
0  (month_month: 600)            17           1
1  (month_month: 601)            10           1
2  (month_month: 602)            13           1
3  (month_month: 603)            10           1

>>> sql("""SELECT regexp_extract(file_path, '[^/]+/[^/]+/[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."orders_by_state_month_part$files" ORDER BY fi…
                                                                                                                                                    file  record_count  file_size_in_bytes
0  data/month_month=2020-01/20261109_123931_00060_ff7zw-b3fa9a24-6cbe-4449-a643-c28f919fc5ae.parquet            17               1040
1  data/month_month=2020-02/20261109_123931_00060_ff7zw-9ca7df27-a8f9-4389-98cd-093efcd05df2.parquet            10                963
2  data/month_month=2020-03/20261109_123931_00060_ff7zw-b6acc66d-d13e-4243-a27d-3ce7a3718731.parquet            13                999
3  data/month_month=2020-04/20261109_123931_00060_ff7zw-444d9432-cd57-4c77-9726-ff743ddcca66.parquet            10                966

>>> sql("""SELECT count(*) AS files, sum(record_count) AS rows FROM lakehouse.gold."orders_by_state_month$files" """)
   files  rows
0      1    50

>>> sql("""SELECT count(*) AS files, sum(record_count) AS rows FROM lakehouse.gold."orders_by_state_month_part$files" """)
   files  rows
0      4    50
```

Figure 24 – Table partitionnée : partitions, fichiers et comparaison avec la table non partitionnée.

Avec un filtre sur février 2020 (Figure 25), la table non partitionnée lit **50 lignes** (80 % filtrées) et la partitionnée **10 lignes** : seul le fichier du mois est lu (élagage de partitions). À 50 lignes il n'y a pas de gain de temps mesurable ; l'intérêt apparaît avec de gros volumes.

```
python — exercice 1 : EXPLAIN ANALYZE (extrait)

>>> print(sql("""EXPLAIN ANALYZE SELECT sum(orders) FROM lakehouse.gold.orders_by_state_month WHERE month = TIMESTAMP '2020-02-01 00:00:00 UTC' """).iloc[0,0])
Fragment 2 [SOURCE]
    CPU: 12.01ms, Scheduled: 95.19ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 50 rows (1.01kB); per task: avg.: 50.00 std.dev.: 0.00, Output: 1 row (9B)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month$data@2562529447020369591, filterPredicate = (month = timestamp(6) with time zone '2020-02-01 00:00:00.00…
          Input: 50 rows (1.01kB), Filtered: 80.00%, Physical input: 1.35kB, Physical input time: 33.94us, Splits: 1, Splits generation wait time: 367.32ms
[…]

>>> print(sql("""EXPLAIN ANALYZE SELECT sum(orders) FROM lakehouse.gold.orders_by_state_month_part WHERE month = TIMESTAMP '2020-02-01 00:00:00 UTC' """).iloc[0,0])
Fragment 2 [SOURCE]
    CPU: 22.60ms, Scheduled: 109.95ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 10 rows (220B); per task: avg.: 10.00 std.dev.: 0.00, Output: 1 row (9B)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month_part$data@4374313399918356290, filterPredicate = (month = timestamp(6) with time zone '2020-02-01 00:00:…
          Input: 10 rows (220B), Filtered: 0.00%, Physical input: 963B, Physical input time: 13.15us, Splits: 1, Splits generation wait time: 111.93ms
[…]
```

Figure 25 – `EXPLAIN ANALYZE` avec filtre sur un mois : sans partition (haut) et avec (bas).

(b) Exercice 2 : cohortes par décennie de naissance

`gold.cohorts` compte, par décennie, les utilisateurs, leurs commandes et la moyenne d'unités par commande (Figure 26). 11 décennies, de 1910 à 2020 ; les totaux (50 utilisateurs, 2 829 commandes) correspondent à silver. La moyenne varie peu (2,79 à 3,19), les cohortes sont petites (1 à 8 utilisateurs), donc peu significatives.

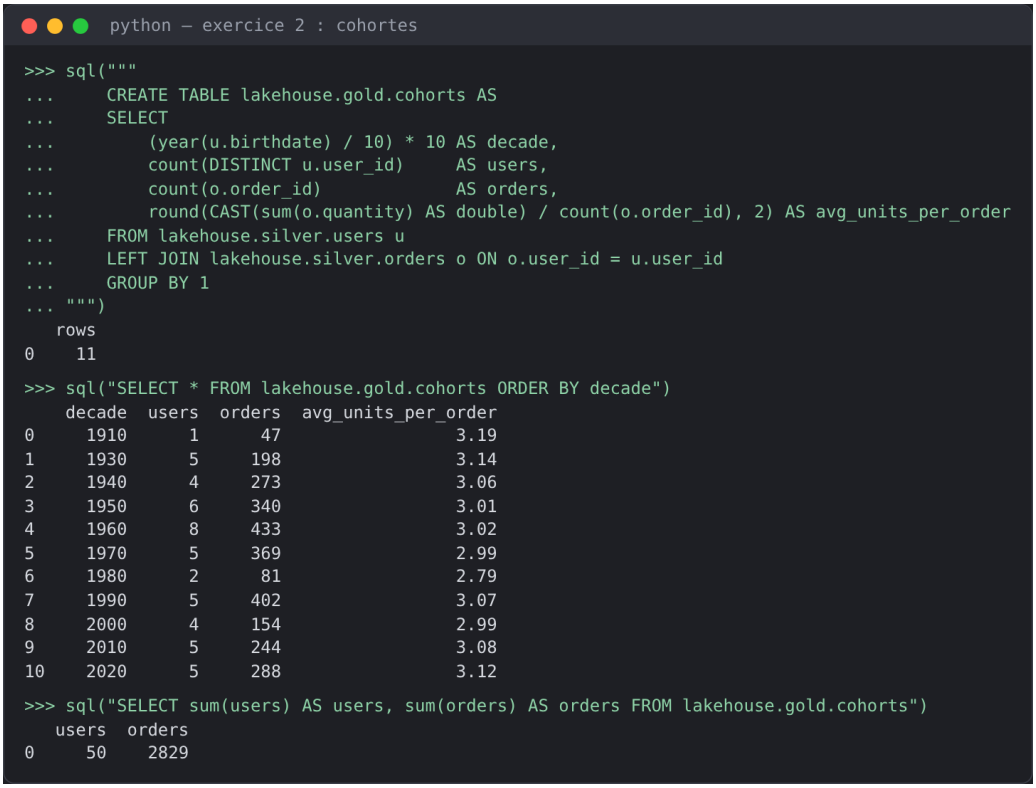


Figure 26 – Table `gold.cohorts`.

(c) Exercice 3 : INSERT, MERGE et time travel

Une commande est ajoutée à `silver.orders` (2 829 devient 2 830), puis `orders_by_state_month` est rafraîchie par `MERGE` : 51 lignes touchées (50 mises à jour, 1 nouvelle combinaison état-mois). La table a ensuite 2 snapshots (`append`, `overwrite`). Le time travel sur le premier donne 2 829 commandes, la table actuelle 2 830 (Figure 27).

```
python — exercice 3 : INSERT, MERGE et time travel

>>> sql("""
...     INSERT INTO lakehouse.silver.orders
...     SELECT '00000000-0000-0000-0000-000000000001', user_id,
...            TIMESTAMP '2020-02-10 12:00:00 UTC', 3, 'croissant'
...     FROM lakehouse.silver.users ORDER BY user_id LIMIT 1
... """)
   rows
0     1

>>> sql("SELECT count(*) AS orders FROM lakehouse.silver.orders")
   orders
0    2830

>>> sql("""
...     MERGE INTO lakehouse.gold.orders_by_state_month AS t
...     USING (
...         SELECT u.state, date_trunc('month', o.ordered_at) AS month,
...                count(*) AS orders, sum(o.quantity) AS units, count(DISTINCT o.user_id) AS buyers
...         FROM lakehouse.silver.orders o JOIN lakehouse.silver.users u ON u.user_id = o.user_id
...         GROUP BY 1, 2
...     ) AS s
...     ON t.state = s.state AND t.month = s.month
...     WHEN MATCHED THEN UPDATE SET orders = s.orders, units = s.units, buyers = s.buyers
...     WHEN NOT MATCHED THEN INSERT (state, month, orders, units, buyers) VALUES (s.state, s.month, s.orders, s.units, s.buyers)
... """)
   rows
0    51

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."orders_by_state_month$snapshots" ORDER BY committed_at""")
         snapshot_id                committed_at  operation
0  2562529447020369591 2026-11-09 12:38:23.232+00:00     append
1  7631086578838822541 2026-11-09 12:39:51.360+00:00  overwrite

>>> snap = sql("""SELECT snapshot_id FROM lakehouse.gold."orders_by_state_month$snapshots" ORDER BY committed_at LIMIT 1""").iloc[0,0]
... print(sql(f"SELECT sum(orders) AS orders_before FROM lakehouse.gold.orders_by_state_month FOR VERSION AS OF {snap}").to_string())
... print(sql("SELECT sum(orders) AS orders_after FROM lakehouse.gold.orders_by_state_month").to_string())
   orders_before
0           2829
   orders_after
0          2830
```

Figure 27 – `INSERT` dans silver, `MERGE` dans gold et lecture de l'état précédent.

(d) Exercice 4 : `optimize`

Sur `top_products` (Figure 28), `optimize` ne change rien : la table n'a qu'un fichier, il n'y a rien à compacter, et aucun snapshot n'est créé (3 avant, 3 après).

```
python — exercice 4 : optimize sur top_products

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at""")
         snapshot_id                committed_at operation
0  5679923997119223469 2026-11-09 12:38:28.711+00:00    append
1  7721050443174576655 2026-11-09 12:38:45.105+00:00    delete
2   433421330162471359 2026-11-09 12:38:48.706+00:00    append

>>> sql("""SELECT regexp_extract(file_path, '[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files" """)
                                                  file  record_count  file_size_in_bytes
0  20261109_123846_00035_ff7zw-a6e69d7d-7512-4da7-a5b0-a90db5d8ae55.parquet             6                 760

>>> sql("ALTER TABLE lakehouse.gold.top_products EXECUTE optimize")
Empty DataFrame
Columns: [rows]
Index: []

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at""")
         snapshot_id                committed_at operation
0  5679923997119223469 2026-11-09 12:38:28.711+00:00    append
1  7721050443174576655 2026-11-09 12:38:45.105+00:00    delete
2   433421330162471359 2026-11-09 12:38:48.706+00:00    append

>>> sql("""SELECT regexp_extract(file_path, '[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files" """)
                                                  file  record_count  file_size_in_bytes
0  20261109_123846_00035_ff7zw-a6e69d7d-7512-4da7-a5b0-a90db5d8ae55.parquet             6                 760
```

Figure 28 – `optimize` sur `top_products` : aucun changement.

Pour voir un effet, je l'ai appliqué à `orders_by_state_month`, réécrite par le `MERGE` (Figure 29). Avant : 3 entrées dans `$files` (le résumé indique 2 fichiers de données ; la troisième est donc, par déduction, le fichier de suppressions écrit par le `MERGE`). Après : un nouveau snapshot de type `replace` et **1 seul fichier** de 51 lignes ; le contenu est inchangé (51 lignes, 2 830 commandes).

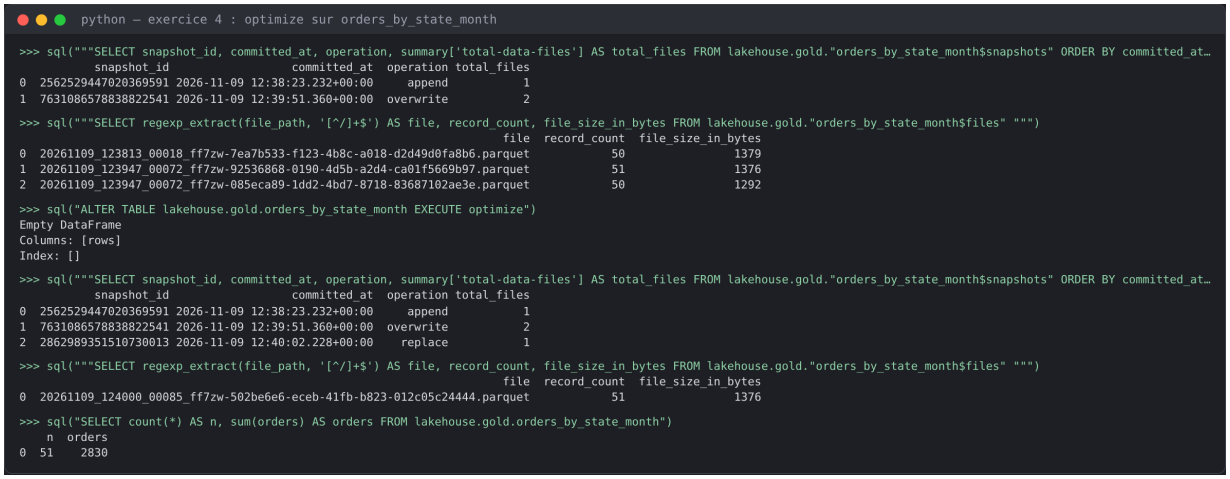


Figure 29 – `optimize` sur `orders_by_state_month` : snapshots et fichiers avant et après.

(e) Exercice 5

Non réalisé : il demande une session Spark connectée à Polaris, que je n'ai pas utilisée pour ces labs.

F. Stockage des tables gold

La Figure 30 liste les fichiers de données de toutes les tables de `gold` dans le bucket. Chaque table a son dossier `<table>-<identifiant>/data/`. `orders_by_state_month_part` a un sous-dossier par mois, ce qui explique ses 4 fichiers. `orders_by_state_month` en montre 4 : le fichier d'origine, les deux écrits par le `MERGE` et celui issu de `optimize`. Les trois premiers restent dans le bucket parce que les anciens snapshots les référencent encore ; Iceberg ne les supprime pas à l'`optimize`.

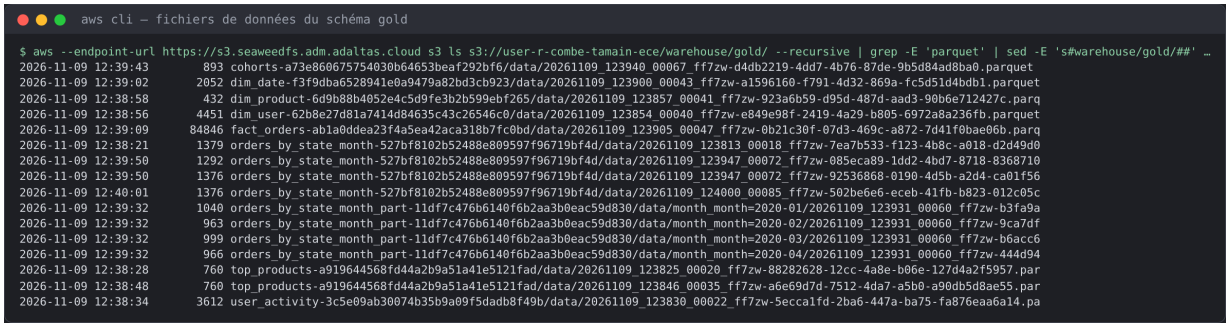


Figure 30 – Fichiers de données du schéma `gold` dans le bucket.

IV. Bilan

- Lab 2 et lab 3 exécutés de bout en bout sur le cluster Adaltas, sans erreur, sauf la comparaison Spark (lab 2) et l'exercice 5 (lab 3), non faits.
- Les données du générateur donnent 2 829 commandes et non 5 000 comme l'annonçaient les labs.
- Points techniques à retenir : jointure `REPLICATED` choisie par Trino pour une dimension de 50 lignes ; l'élagage de partitions divise par 5 les lignes lues sur l'exemple (50 contre 10) ; `DELETE` puis `INSERT` ouvre une fenêtre à table vide (3,6 s ici), contrairement à `MERGE` ; les clés `row_number()` changent à la reconstruction et cassent le lien avec le fait.

V. Annexes

A. Catalogue Trino

Listing 1 – Catalogue `lakehouse` dans les valeurs du chart Helm de Trino (extrait)

```
trino:
  additionalCatalogs:
    lakehouse: |-
      connector.name=iceberg
      iceberg.catalog.type=rest
      iceberg.rest-catalog.uri=${ENV:POLARIS_URI}/api/catalog
      iceberg.rest-catalog.warehouse=${ENV:POLARIS_CATALOG}
      iceberg.rest-catalog.security=OAUTH2
      iceberg.rest-catalog.oauth2.server-uri=${ENV:POLARIS_URI}/api/catalog/v1/oauth/tokens
      iceberg.rest-catalog.oauth2.credential=${ENV:POLARIS_DEV_CREDENTIAL}
      iceberg.rest-catalog.oauth2.scope=PRINCIPAL_ROLE:ALL
      fs.native-s3.enabled=true
      s3.endpoint=${ENV:S3_ENDPOINT}
      s3.region=us-east-1
      s3.path-style-access=true
```

B. Scripts Spark

Listing 2 – silver_spark.py : couche silver du lab 07

```
"""Lab 07 : couche silver avec PySpark, ecrite en tables Iceberg (catalogue hadoop 'lab')."""
import os
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from spark_common import session, BUCKET

spark = session("lab-silver-pyspark")
users_bronze = spark.read.csv(f"s3a://{BUCKET}/bronze/users.csv", header=True, inferSchema=True, samplingRatio
    =0.01, multiLine=True)
orders_bronze = spark.read.csv(f"s3a://{BUCKET}/bronze/orders.csv", header=True, inferSchema=True, samplingRatio
    =0.01, multiLine=True)

users_typed = (
    users_bronze.select(
        F.col("uuid").cast("string").alias("user_id"),
        F.trim("username").alias("username"),
        F.trim("name").alias("name"),
        F.upper(F.trim("sex")).alias("sex"),
        F.lower(F.trim("mail")).alias("email"),
        F.col("birthdate").cast("date").alias("birthdate"),
        F.split(F.col("address"), "\n").alias("address_lines"),
    )
    .withColumn("street", F.element_at("address_lines", 1))
    .withColumn("address_line_2", F.element_at("address_lines", 2))
    .drop("address_lines")
)
stg_users = users_typed.select(
    "user_id", "username", "name", "sex", "email", "birthdate", "street",
    F.regexp_extract("address_line_2", "^(.+), [A-Z]{2} [0-9]{5}$", 1).alias("city"),
    F.regexp_extract("address_line_2", "([A-Z]{2}) ([0-9]{5})$", 1).alias("state"),
    F.regexp_extract("address_line_2", "([A-Z]{2}) ([0-9]{5})$", 2).alias("zip_code"),
)
stg_users = stg_users.withColumn("city", F.when(F.col("city") == "", None).otherwise(F.col("city")))

orders_typed = orders_bronze.select(
    F.col("uuid").cast("string").alias("order_id"),
    F.col("user_uuid").cast("string").alias("user_id"),
    F.col("date").cast("timestamp").alias("ordered_at"),
    F.col("quantity").cast("int").alias("quantity"),
    F.lower(F.trim("product")).alias("product"),
)
print("Original:", orders_typed.count())
orders_with_duplicates = orders_typed.unionByName(orders_typed.limit(3))
print("With duplicates:", orders_with_duplicates.count())
w = Window.partitionBy("order_id").orderBy(F.col("ordered_at").desc())
```

```
stg_orders = orders_with_duplicates.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
print("After deduplication:", stg_orders.count())

spark.sql("CREATE NAMESPACE IF NOT EXISTS lab.silver")
stg_users.writeTo("lab.silver.users").createOrReplace()
stg_orders.writeTo("lab.silver.orders").createOrReplace()
users_iceberg = spark.table("lab.silver.users")
orders_iceberg = spark.table("lab.silver.orders")
assert users_iceberg.count() == users_bronze.count()
assert orders_iceberg.count() == stg_orders.count()
print("SILVER OK users=%d orders=%d" % (users_iceberg.count(), orders_iceberg.count()))
spark.sql("SELECT snapshot_id, operation, committed_at FROM lab.silver.orders.snapshots").show(truncate=False)
spark.stop()
```

Listing 3 – register_polaris.py : enregistrement dans Polaris (lab 09)

```
"""Lab 09 : enregistre silver.orders dans Polaris sans copier les donnees, cree silver.users via Polaris."""
import os, re, subprocess
from spark_common import session, BUCKET

AWS = os.environ.get("AWS_CMD", "aws --profile default")
meta = f"s3://{BUCKET}/warehouse/silver/orders/metadata/"
version = subprocess.run(f"{AWS} s3 cp {meta}version-hint.text -", shell=True, capture_output=True, text=True,
     check=True).stdout.strip()
print("version-hint:", version)
spark = session("lab-polaris", polaris_credential=os.environ["POLARIS_DEV_CREDENTIAL"])
spark.sql("CREATE NAMESPACE IF NOT EXISTS polaris.silver")
if spark.catalog.tableExists("polaris.silver.orders"):
    print("silver.orders deja enregistre dans Polaris")
else:
    spark.sql(f"""
        CALL polaris.system.register_table(
          table => 'silver.orders',
          metadata_file => 's3a://{BUCKET}/warehouse/silver/orders/metadata/v{version}.metadata.json')
    """).show(truncate=False)
spark.table("lab.silver.users").writeTo("polaris.silver.users").createOrReplace()
spark.sql("SELECT count(*) AS orders FROM polaris.silver.orders").show()
spark.sql("SELECT count(*) AS users FROM polaris.silver.users").show()
spark.sql("SHOW TABLES IN polaris.silver").show()
spark.stop()
```